

Security Audit

YOM 2nd (DePIN)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The YOM team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

YOM is a DePIN (decentralized physical infrastructure network) focused on cloud gaming. By transforming idle gaming PCs into pixel-streaming nodes, it enables low-latency, cost-effective game streaming of Unreal Engine 5 titles directly to any device or platform—including web, apps, and Telegram—without reliance on centralized cloud infrastructure. Node operators earn YOM tokens (with APY ranging from 72% to 480%) by contributing compute power, while studios benefit from scalable, near-zero-cost deployment and built-in monetization tools. With partnerships across game studios, blockchain firms, and cloud providers, YOM is building a distributed, developer-friendly ecosystem that aims to revolutionize how games are delivered and monetized on-chain.

Project Name: YOM

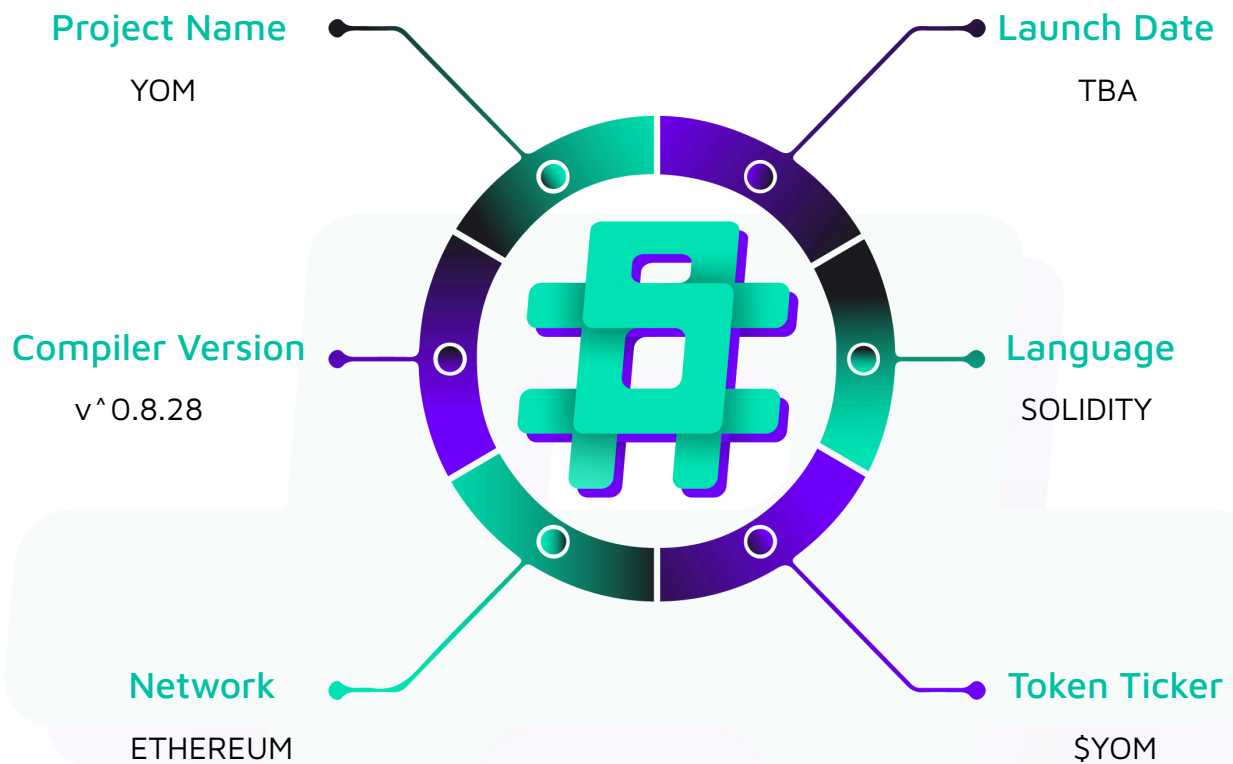
Project Type: DePIN

Compiler Version: ^0.8.28

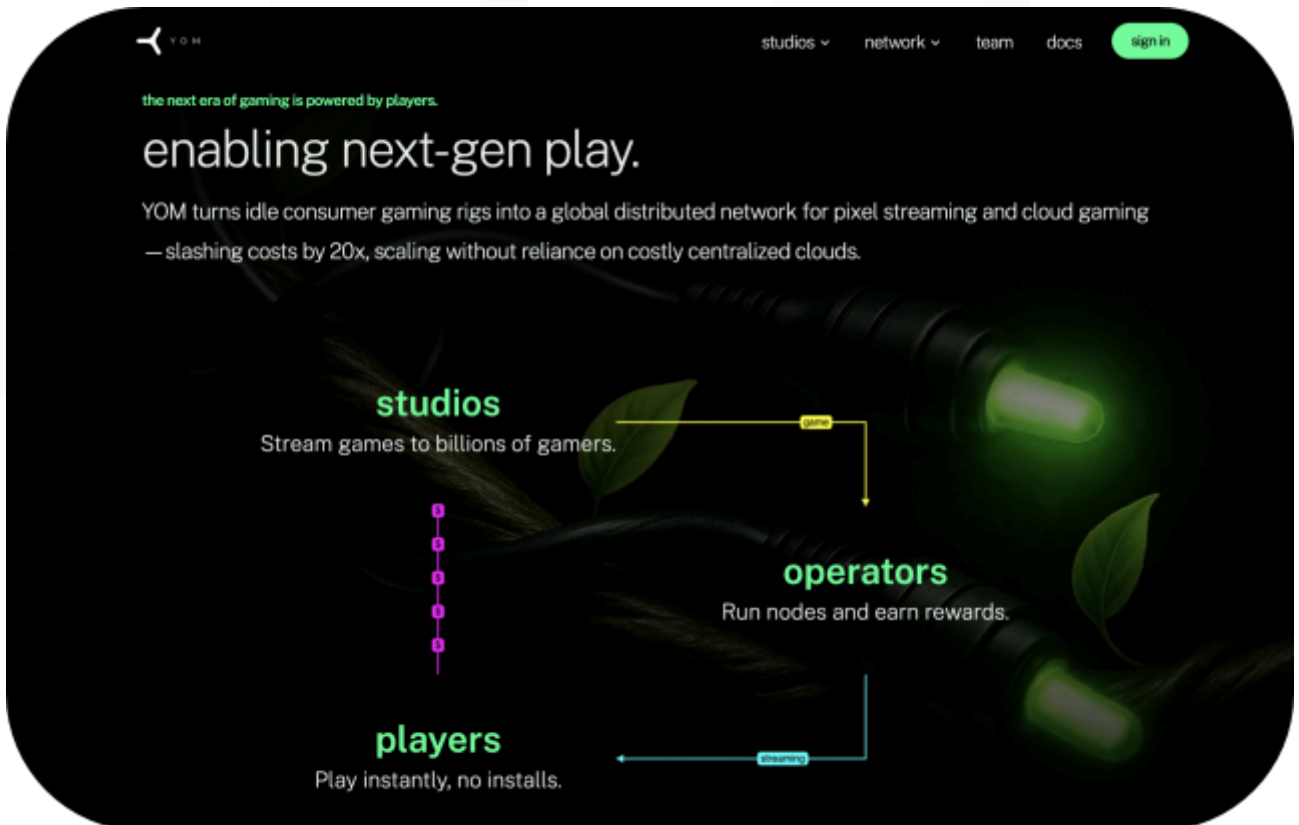
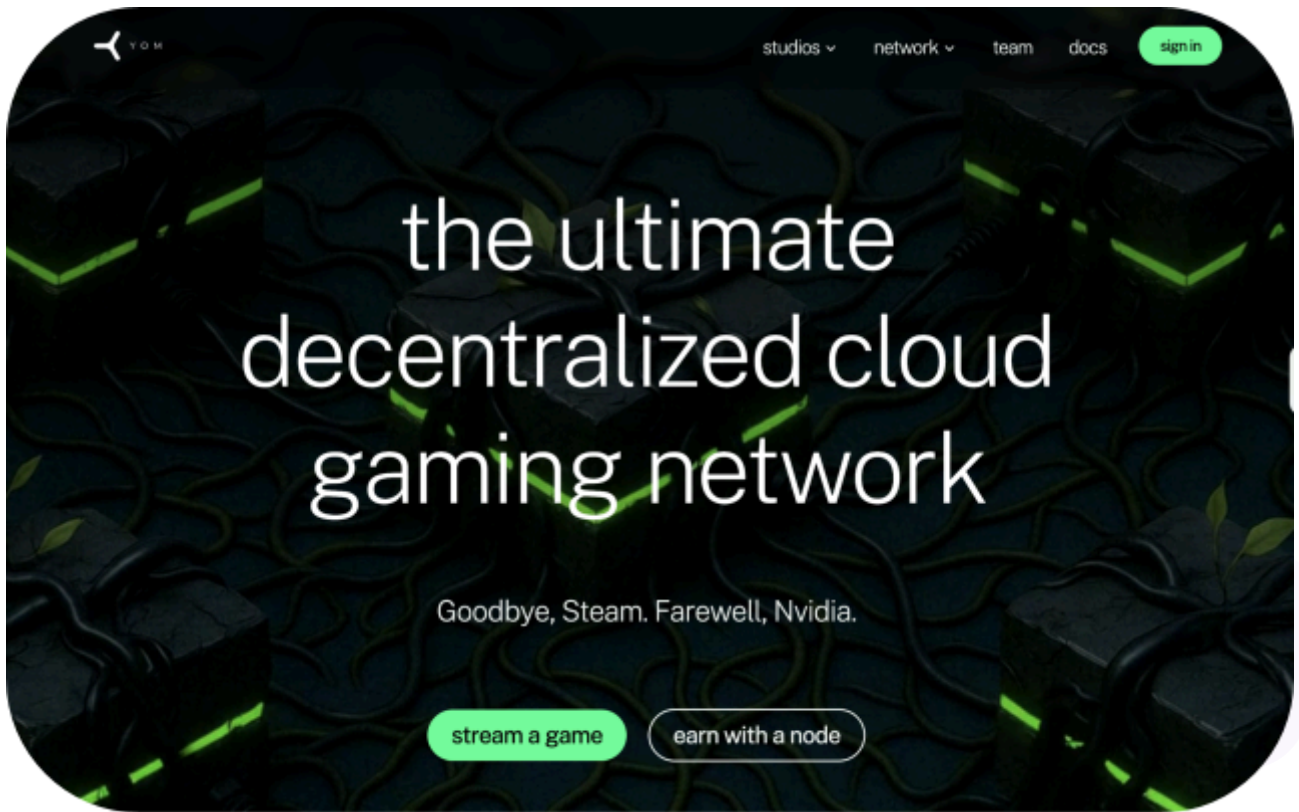
Website: <https://www.yom.net/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the YOM project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	YOM Smart Contracts
Platform	Ethereum / Solidity
Audit Date	July, 2025
Contract 1	Factory.sol
Contract 2	Lock.sol
Contract 3	YOM.sol
Audited GitHub Commit Hash	a130e95527817c182090c7ac1c22bf213ff760ea
Fix Review GitHub Commit Hash	9bf94bc4b3cc8f2061037a5da840b82df3cb5f7b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section, and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Low severity vulnerability

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Lock.sol Allows users to: - Deploy a time-locked ETH vault by sending ETH during contract creation - Set a future unlock timestamp that determines when funds can be withdrawn Allows the owner to: - Withdraw all stored ETH after the specified unlock time has passed - Receive withdrawal confirmation through the Withdrawal event	Contract achieves this functionality.
Factory.sol Allows users to: - Deploy contracts with deterministic addresses using the CREATE2 opcode - Precompute the exact deployment address before deploying a contract - Deploy any contract by providing its creation bytecode and a custom salt value	Contract achieves this functionality.
Yom.sol Allows users to: - Transfer YOM tokens locally between addresses (tax-free for regular transfers)	Contract achieves this functionality.

- Buy YOM from configured DEX pools (subject to 2% buy tax)
- Sell YOM to configured DEX pools (subject to 3% sell tax)
- Send YOM cross-chain to other blockchains via the LayerZero OFT protocol (tax-free)
- Receive YOM from other chains through the LayerZero bridge
- Burn their tokens or approved tokens from other addresses
- Approve spending allowances following the ERC20 standard

Allows admins to:

- Set buy and sell tax rates (maximum 10% each)
- Configure which DEX pair addresses trigger tax collection
- Exclude specific addresses from all tax obligations
- Freeze individual accounts to prevent all token movements
- Pause the entire contract to halt all transfers
- Change the fee collector address that receives tax revenue
- Configure cross-chain settings, including supported destination chains
- Rescue accidentally sent ERC20 tokens or ETH from the contract
- Set MEV protection cooldown periods for tax collection
- Batch configure multiple addresses for exclusions, freezing, or AMM pairs

Code Quality

This audit scope involves the smart contracts of the YOM project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the YOM project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] YOM - Composing is not implemented

Description

The OFT contract has a function called `send` that is responsible for sending composed messages that can be executed by the composer.

```
function send(
    SendParam calldata _sendParam,
    MessagingFee calldata _fee,
    address _refundAddress
) external payable virtual returns (MessagingReceipt memory msgReceipt, OFTReceipt
memory oftReceipt) {
    // @dev Applies the token transfers regarding this send() operation.
    // - amountSentLD is the amount in local decimals that was ACTUALLY
    sent/debited from the sender.
    // - amountReceivedLD is the amount in local decimals that will be
    received/credited to the recipient on the remote OFT instance.
    (uint256 amountSentLD, uint256 amountReceivedLD) = _debit(
        msg.sender,
        _sendParam.amountLD,
        _sendParam.minAmountLD,
        _sendParam.dstEid
    );

    // @dev Builds the options and OFT message to quote in the endpoint.
    (bytes memory message, bytes memory options) = _buildMsgAndOptions(_sendParam,
    amountReceivedLD);

    // @dev Sends the message to the LayerZero endpoint and returns the LayerZero
    msg receipt.
    msgReceipt = _lzSend(_sendParam.dstEid, message, options, _fee,
```

```
_refundAddress);  
    // @dev Formulate the OFT receipt.  
    oftReceipt = OFTReceipt(amountSentLD, amountReceivedLD);  
  
    emit OFTSent(msgReceipt.guid, _sendParam.dstEid, msg.sender, amountSentLD,  
amountReceivedLD);  
}
```

In order to execute it, the contract should implement the `IzCompose` interface. Since the current implementation does not implement it, any composed message leads to gas wastage, without an immediate security risk.

Recommendation

Consider disabling the functionality by overriding it.

Status

Resolved

QA

[Q-01] Lock - Remove unnecessary comments

Description

The withdraw function contains unnecessary comments indicating that the codebase is still in the testing stage.

```
function withdraw() public {  
    // Uncomment this line, and the import of "hardhat/console.sol", to print a log  
    // in your terminal  
    // console.log("Unlock time is %o and block timestamp is %o", unlockTime,  
    block.timestamp);  
  
    require(block.timestamp >= unlockTime, "You can't withdraw yet");  
    require(msg.sender == owner, "You aren't the owner");  
  
    emit Withdrawal(address(this).balance, block.timestamp);  
  
    owner.transfer(address(this).balance);  
}
```

Recommendation

Consider removing the comments once the codebase is production-ready.

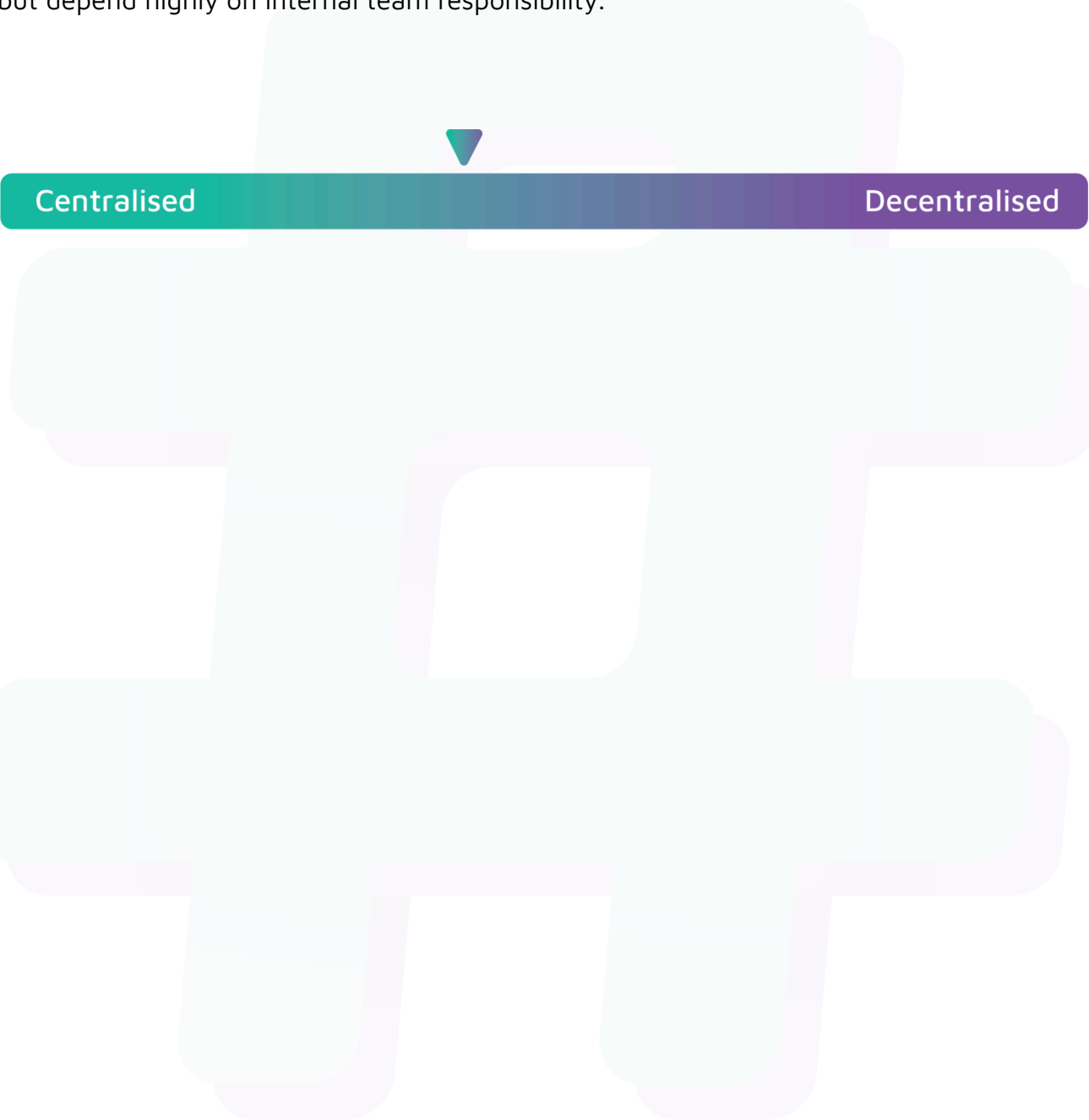
Status

Resolved

Centralisation

The YOM project values security and utility over decentralisation.

The owner executable functions within the protocol increases security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the YOM project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.